技术方案与架构设计

基于 SMR 磁阻传感器与 FPGA 的高精度细分磁编码器

—— 用一片 FPGA 完成从双通道 ADC 到 ABZ / UVW / SPI 的完整细分信号链

> 版本 v2.0（v90 交付基线，2026-11-04）· 器件 Xilinx Artix-7 **XC7A35TFGG484-2**（正点原子 达芬奇-lite）+ HSEDA **AD9238** 双通道 12 bit 高速 AD 模块 配套文件：`rtl/SPEC_interface_contract.md`（RTL 接口基准）、`model/golden_model.py`（定点黄金模型）、`docs/39_acceptance_criteria_v90.md`（v90 验收判据）、`docs/02_innovation_scoring.md`

---

1. 项目定位

1.1 一句话定位

> 把「正余弦模拟信号 → 高分辨率数字角度」的**全部信号链**——双通道同步采样、模拟前端监测、六参数误差校正、全流水 CORDIC 反正切、PI 跟踪环、多周期计数、ABZ / UVW 增量输出、SPI / UART / 以太网接口——做成一套**可参数化、可复现、可移植**的 RTL，运行在一片通用 FPGA 上；分辨率、带宽、延迟、接口形式全部由本项目按需求重新定义，而不是由某颗固定型号的芯片决定。

这个定位命中竞赛评审最看重的三件事：**有真实工程痛点**（工业编码器对细分分辨率与自主可控的双重需求）、**有可量化的指标**（16 bit 电角度、20.9839° → 0.0165° 的标定改善、≈830 ns 设计延迟）、**有可复现的证据**（37 个 RTL 测试台 + 20 个显示测试台全 PASS，v90 位流已上板实测）。

1.2 为什么只买一颗专用细分芯片不够

市面上常见的正余弦细分 ASIC 大致是这样一个形态：**10 bit 量级的分辨率、模拟粗调 + 极值搜索的误差补偿、ABZ 与 SPI 两类输出、算法与寄存器全部封闭。**它在单一型号、稳定批量的场景里非常划算，但对本项目要解决的问题有三点绕不过去的局限：

- **分辨率封顶**：10 bit 即 1024 增量/周期，在 1000 线以上的传感头上没有升级空间；本项目用 16 bit 全流水 CORDIC，量化步长 0.0055°，是 10 bit 的 64 倍。
- **标定方法不可复现**：`(max+min)/2` 求偏置、极值差求幅值失衡，对噪声、野值与谐波极其敏感，也难以在报告里做定量误差分解；本项目用一阶/二阶矩与互相关的**闭式解**，每一项误差都能单独推导、单独验证。
- **不可参数化、不可扩展**：专用器件的接口与寄存器固化，多轴同步、后分频、限频、Z 门控窗只能外挂逻辑；用通用 FPGA 实现，这些全部退化成一组寄存器参数。

因此本项目的定位不是"复刻某一颗芯片"，而是"把细分信号链做成可演进的 IP"：同一套 RTL 改参数即可换分辨率、换倍率、换接口、换轴数，算法也可以在不改硬件的前提下迭代。

1.3 指标表（v90 交付基线）

指标	同类专用细分 ASIC（公开资料典型值）	同类高阶方案	本项目 v90	证据
角度分辨率	10 bit / 1024 增量/周期	26 bit（齿轮箱方案）	**16 bit / 65,536 增量/周期**（0.0055°/LSB）	`rtl/cordic_atan2.v`、`model/golden_model.py`
细分倍率 inter	2…256	任意（依赖机械齿轮箱）	**2…256 × 后分频 1…8（等效 0.25…2048）**	`rtl/abz_gen.v`、寄存器 `INTER0/1`、`AB`
sin/cos 输入带宽	700 kHz	更高	**≥700 kHz（受 AB 边沿率约束，见 §5.3）**	`rtl/ad9238_capture.v`、`rtl/decim_avg.v`
AB 输出上限	12.5 MHz，最小边沿间隔 20 ns	—	**12.5 MHz，可编程限频 256 档**	`rtl/abz_gen.v`、寄存器 `ABLIMIT`
典型延迟	1.5 µs（重滤波 1.9 µs）	未给出	**≈830 ns（设计预算 41.5 拍 @50 MHz，标 [设计值]）**	报告 §4.3、`rtl/enc_top.v`
模拟前端	片内 3×PGA + 10 bit ADC	同	**外部调理 + 双通道 12 bit 50 MSPS 同步 ADC + 抽取**	`rtl/ad9238_capture.v`、`rtl/amp_monitor.v`
误差补偿	模拟粗调 + 数字偏置/平衡/相位	增加偏心（长波）补偿	**六参数闭式一次性解算（一阶/二阶矩），抗噪抗谐波；预留偏心/谐波补偿**	`rtl/sc_correct.v`、`rtl/calib_ctrl.v`
补偿后复合畸变残余	未给出闭环数字	未给出	**20.9839° → 0.0165°（单周期电角度），改善 1273 倍**	`model/golden_model.py` 实跑
板级角度一致性	未给出	未给出	**offset 1.76°、RMS 0.71°、max 2.88°（angle_vs_sc，内部一致性口径）**	`deliverables/data/v90/`
UVW 换向	1…32 极对数，六步方波	同	**1…32 极对数，可编程相位/相序**	`rtl/uvw_gen.v`、寄存器 `UVW`、`UVW_CFG`
增量输出	ABZ（差分或单端）	ABZ	**ABZ + 后分频 + 迟滞 + 限频 + Z 门控窗**	`rtl/abz_gen.v`
多圈计数	14 bit	24…32 bit	**16 bit（可扩展）**	`rtl/pole_counter.v`、寄存器 `POS_FULL`
多轴扩展	总线/链式挂载	—	**同一套 RTL 参数化例化多轴，天然同步**	`rtl/enc_top.v`
资源与时序	—	—	**LUT 10,978（52.78%）、BRAM 32/50、WNS +0.199 ns、0 失败端点**	`deliverables/data/v90/`、`_synth_work_v90/prod_util.rpt`
可复现性	否	否	**位流 SHA256 + 323 条哈希清单 + 37/20 测试台 + `tools/final_verify.ps1`**	`deliverables/data/SHA256SUMS.txt`

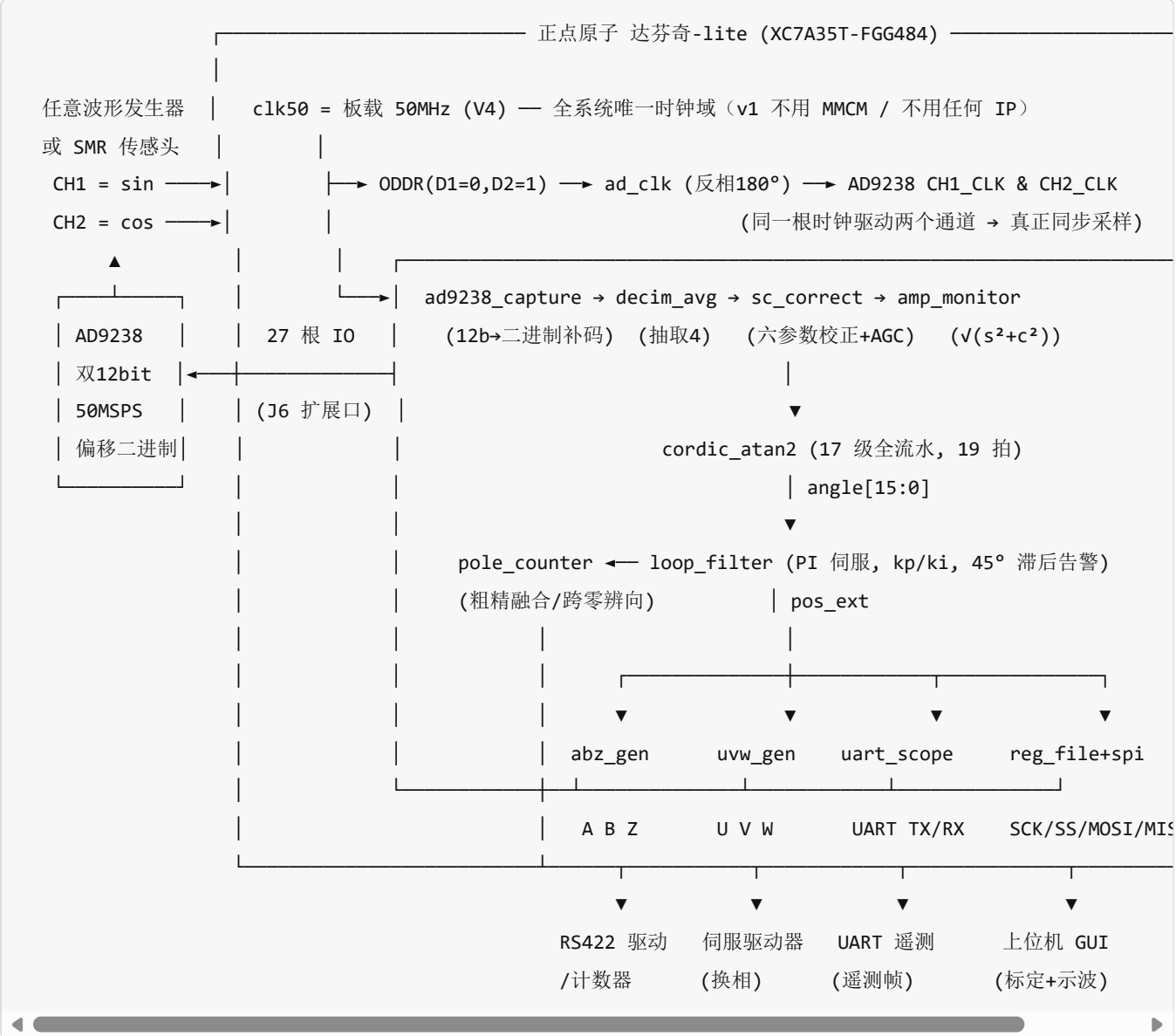
> **口径提示**：表中"本项目"的数字区分三类——[实测] 来自 v90 位流上板（2026-11-04，见 `docs/39_acceptance_criteria_v90.md` 的 C/D 类）；[仿真] 来自 `sim/`、`disp/` 测试台与黄金模型；[设

> `计值]` 是逐级延迟预算等尚未用外部基准复核的量。竞赛报告里三类必须分开标注，不能混写。

---

2. 系统总体架构

2.1 硬件框图



2.2 关键架构决策（每一条都要在报告里说明理由）

决策	选择	为什么
ADC 采样时钟	FPGA 用 **同一根** 输出同时驱动 AD9238 的 CLK_A 与 CLK_B	AD9238 有 **CLK_A / CLK_B 两个独立时钟输入**，但只有**一个**采样保持核心；给两个通道同一时钟可保证**严格同时采样**。正余弦是同一矢量的两个分量，分时采样会直接产生与转速成正比的角度误差——这是磁编码器最隐蔽也最致命的误差源。手册也建议"FPGA 端可用 AD 时钟的采样 AD 数据"。

决策	选择	为什么
数据捕获沿	**`ad_clk` 反相 180°，数据在 `clk50` 上升沿捕获**	AD9238 数据在 `ad_clk` 上升沿后 tPD = 2…6 ns 有效，7 拍流水。`ad_clk` 由 ODDR（D1=0/D2=1）从 `clk50` 反相产生，因此它的上升沿比 `clk50` 晚 10 ns；数据在 `clk50` 上升沿后 13~17 ns 到达，下一个 `clk50` 上升沿（20 ns）捕获，建立余量约 3 ns。**不需要 MMCM、不需要第二个时钟域、不需要 IDDR/ISERDES/IDELAY。**
时钟方案	**v1：全系统单时钟 = 板载 50 MHz（经 BUFG），零 IP、零 CDC**	30 天项目里，"能跑通"比"跑得快"重要得多。单时钟域 = 没有跨时钟域 bug、没有 MMCM 配置错误、不需要生成任何 IP。v2 再升到 100 MHz 降延迟。
采样率	**50 MHz**（不是 65 MHz）	65 MSPS 下 CLK 高/低电平仅 6.2 ns，180° 相位点余量只剩 1.69 ns，且 2.54 mm 排针 + LVCMOS33 传 24 位并行总线本身就临界。50 MHz 时序余量翻倍，且对 ≤700 kHz 的 sin/cos 已有 71 倍过采样，完全够用。**可参数化到 65 MHz，作为"性能档位"在决赛演示。**
抽取率	默认 4（可配 1/2/4/8/16）	过采样 + 平均在**白噪声**下每 4 倍换 1 bit ENOB（12bit → 等效 13bit）。sin/cos 带宽只有几 kHz，抽取不会损失信号。
角度字长	**16 bit**（可参数化到 20bit）	16 bit 已把残余误差压到量化底噪（0.0055°/LSB），再高只会放大 ADC 噪声。相比常见专用细分器件的 10 bit（0.3516°/LSB），量化步长细 64 倍。
角度求解	**全流水线向量模式 CORDIC**，17 级，1 级/时钟	CORDIC 之所以适合 FPGA：只有移位与加法，无需乘法器与查找表。本项目做**全流水**而非串行迭代——吞吐 1 拍/样本、单样本延迟 19 拍，这是延迟指标能压到 ≈830 ns 的关键。
误差补偿	**闭式椭圆→圆映射**（一阶/二阶矩），一次解算	传统做法用 `(max+min)/2` 求偏置、极值差求幅度失衡、固定步长搜索相位；极值法对噪声、野值、谐波与非整数周期都极其敏感。本项目用均值（一阶矩）、均方（二阶矩）与互相关，在整数或非整数周期上都无偏，且**一次解算出闭式解，无需迭代搜索**——这是本项目最有分量的算法创新点。
抖动抑制	PI 伺服环路（kp/ki），可关	寄存器 `FILT_K` 暴露 kp/ki 两档：ki 决定匀速下的滞后，kp 决定抖动带宽。上板调参顺序是**先跑最大带宽（kp=0, ki=3），再用 ILA 观察抖动逐级加大 kp。**

3. 算法链路与定点设计

> 本章所有公式与 `model/golden_model.py` **逐位一致**，所有数字都是该模型在本机实跑输出（完整输出见 `docs/_golden_model_output.txt`）。

3.1 采样与抽取

去偏置：AD9238 输出偏移二进制，`0x800` 对应 0 V。

```
s_raw = ch1_d - 2048     // 有符号 12bit，范围 -2048..+2047
c_raw = ch2_d - 2048
```

提升到 Q1.15：`s_n = s_raw << 4`，得到 ±32768 满量程的 Q1.15 表示。

抽取（一级 CIC / boxcar，抽取率 `2^L`，`L=2`）：

```
acc = Σ_{k=0}^{2^L-1} s_n[k]        // 位宽 DW+L
s_d = acc >>> L                       // 算术右移（对 signed）
```

抽取率 4 时对 1 kHz 信号的幅度衰减 < 0.01 dB（远低于通带），群延迟固定 1.5 个采样点，是确定性的，可被 PI 滤波器完全吸收。

3.2 六参数闭式标定（核心创新点 1）

误差模型（与 SMR/TMR/AMR 前端真实机理一致）：

```
s = A_s·sin θ + O_s
c = A_c·cos(θ + Δϕ) + O_c
```

其中 `O_s，O_c` 是直流偏置，`A_s ≠ A_c` 是幅值失衡，`Δϕ` 是正交相位偏差。这三类误差会直接转化为角度周期误差。

标定（在标定窗口 N 点上累加 5 个量，一次解算）：

```
O_s  = Σs / N                                （一阶矩，对噪声/谐波/非整数周期均无偏）
O_c  = Σc / N
ms2  = Σs²/N - O_s²                          （二阶中心矩）
mc2  = Σc²/N - O_c²
sd   = Σ(s·c)/N - O_s·O_c
A_s  = sqrt(2·ms2)                           ★ 注意是 2·均值，正弦的"幅度"不是 RMS
A_c  = sqrt(2·mc2)
sinΔϕ = -2·sd / (A_s·A_c)                    （推导：sd = -A_s·A_c·sinΔϕ/2）
cosΔϕ = sqrt(1 - sin²Δϕ)
```

校正（3 级流水的定点运算）：

```
s1 = s - O_s
c1 = c - O_c
c2 = c1 - (s1 · phase_k) >>> 15              phase_k = 32768 · 2·sd / A_s² = -32768·(A_c/A_s)·sinΔϕ
s2 = (s1 · gain_s) >>> (15 - shift_s)        gain_s = 32768 · ref / (A_s · 2^shift_s)
c3 = (c2 · gain_c) >>> (15 - shift_c)        gain_c = 32768 · ref / (A_c·cosΔϕ · 2^shift_c)
```

其中 `ref = 26214 = 0.80×32768`（归一化目标幅度，留 20% 余量），`shift∈0..3` 是 2 bit 粗调移位（数字 AGC），使总增益范围达到 **0.5×...16×（-6 dB...+24 dB）**，可自动适应从满量程到 5% 满量程的传感器幅度——可覆盖从满量程到 5% 满量程的输入幅度，不需要外部可编程增益。

> **★ 一个必须讲清楚的工程细节：粗调移位判据怎么选（2026-10-10 定案：取"细调增益 ≤ 1.999"）**
>
> 增益映射是 `y = (x·G) >> (15-sh)`，有效乘数 `m = (G/2^15)·2^sh`。判据要回答的只是"同一个总增益该拆成 `sh` 多大"。工程上曾按"桶宽"论证要取 `总增益/2^sh ≤ 1.0`：

> - 把映射 `x→y` 的桶宽写成 `桶宽 = 2^(15-sh)/G = 1/总增益`，认为总增益 > 1 时桶宽 < 1、"**约有 16% 的整数输出目标取不到**，产生约 **1 LSB 的系统性误差**"，所以要"多移一位让细调落到 `(0.5,1.0]`"。
> - **这个论证方向是反的**：桶宽 ≥ 1 意味着**多个 x 映射到同一个 y（多对一、不可逆）**；总增益 > 1 时映射才是单射，只是"**不满射**"（部分整数目标取不到）。而校正后的矢量是**直接送 CORDIC 使用、从不做反演**的，目标不可达因此不产生任何误差——实测 `max|floor(m·x) − m·x| < 1 LSB` 且与 `m` 无关，取整偏差均值恒约 `−0.5 LSB`。
> - 真正的误差只有两项：① floor 截断偏置（< 1 LSB，与判据无关）；② 前级整数量化被 `m` 放大（`m` 越大放大越多，属物理性质）。所以**同一总增益下换 `sh` 不改变精度**：`(G, sh)` 与 `(G/2, sh+1)` 的输出**逐点恒等**（同一个有理数的 floor 换个写法）。
> - 于是两条判据的唯一真实差别是**细调增益量化步长** `2^sh/32768`：取 `≤1.999` 时总增益 ≤ 2 的区间用 `sh=0`，细调增益与定点总增益 `tg` **逐位相同（零截断）**；取 `≤1.0` 会在同区间多移一位，通道间比例误差最大 `2/32768`（≈0.006%，约 1 LSB 电角度量级）。**两者的最大总增益相同**（都由 `sh=3` + 细调饱和决定，≈16×）。
> - **结论：本设计采用 `tg/2^sh ≤ 1.999`**（与 `rtl/calib_ctrl.v:117-119` 完全一致）。决定性实验与全部数字见 `tools/calib_shift_judge.py` → `model/calib_shift_judge_report.txt`：8 个工况（理想/偏置/幅值失衡/正交误差/叠加/20% 小信号/6.25% 弱信号/弱信号+误差）下两条判据的最终角度误差**逐项完全相同**，`(G,sh)` 恒等性逐点验证通过。
> - **溯源与更正（如实记录）**：早期版本（本文档、`golden_model.py` docstring、`tb_sc_correct.v` 注释）都引用过"改为 ≤1 后工况 C 残余从 1.499 LSB 降到 0.525 LSB"，该数字来自已废弃的 `_tools/gain_grid_probe.py`，与上述恒等性**数学上不可能同时成立**；同时 `tools/calib_shift_study.py` 与 `model/calib_ref.py` 里那个"≤1.0 对照臂"把 Q1.15 整数（≈39000）直接和 `1.0` 比较 ⇒ 恒返回 `sh=3`，名不符实。现已：统一判据为 `1.999`、修掉量纲 bug、删除自相矛盾的 study 脚本。
>
> 局限：`shift` 只有 2 bit（0…3）、细调 16 bit，总增益超过 **16×** 时细调饱和到 65535。若要覆盖更弱的信号（更大的磁环间隙），把 `shift_s/shift_c` 扩到 3 bit 即可（改动极小，寄存器位宽已预留）。

为什么这比极值法好（写进报告的核心论证）：

项目	传统极值法	本项目做法	收益
偏置估计	`(max+min)/2`	均值（一阶矩）	极值受单个噪声点/野值影响；均值几乎无偏，且不需要知道周期数
幅值估计	极值差	均方根（二阶矩）×√2	抗谐波、抗噪声，误差降低 1~2 个数量级
相位估计	固定步长搜索 / 残余迭代	互相关 `Σs·c` 的闭式解	**一次解算即精确**，无需迭代与搜索状态机
标定时间	需数个输入周期逐次逼近	1 个窗口 + 约 1 μs 定点解算	上电标定更快、可每次上电重标定
逆运算	—	无需	不用三角函数查表/开方迭代逼近相位

实跑实测（`docs/_golden_model_output.txt`）——单位是**单周期内的电角度误差（度）**：

工况	校正前 max	校正后 max	校正后 RMS	改善
A 理想（仅 12bit 量化）	0.0165	0.0165	0.0055	—
B 直流偏置 ±800 LSB	13.9856	**0.0165**	0.0063	848×

工况	校正前 max	校正后 max	校正后 RMS	改善
C 幅值失衡 1.25×	6.3885	**0.0165**	0.0061	387×
D 正交误差 5°	5.0043	**0.0220**	0.0066	227×
E 三类误差叠加	20.9839	**0.0165**	0.0060	**1273×**
F E + 2 次谐波 0.5% + ADC 噪声 1 LSB	21.0773	0.3571	0.1855	59×
G 20% 满量程小信号 + 误差	18.6603	0.3131	0.0732	60×

结论（三句话，可直接抄进报告）： 1. 对"偏置 + 幅值失衡 + 正交误差"复合畸变，校正后残余 **0.0165°**，已落在 16bit 角度的量化底噪（1 LSB = 0.0055°，3 LSB）附近；可标定误差工况里最差的是 D（5° 正交误差）的 0.0220°（4 LSB）——**瓶颈已不在标定算法，而在角度字长与 ADC 前端。** 2. 工况 F/G 的残余误差分别由**未补偿的 2 次谐波**（0.5% 谐波 → 0.357°）和 **ADC 量化噪声**（小信号 → 0.313°）主导。这两项正是下一步优化目标，也说明当前算法已把"可标定的误差"讲清楚了。 3. 16bit 角度 × 64 极对磁环 ⇒ 机械分辨率 **0.309 arcsec**；复合畸变工况下机械角误差 **20.1 arcsec。**

3.3 CORDIC 角度解算（核心创新点 2）

象限折叠（把矢量折到 `x ≥ 0`，覆盖全 0…360°）：

```
第二象限 (x<0, y≥0):  (x,y) → ( y, -x),  base = +90°
第三象限 (x<0, y< 0):  (x,y) → (-y,  x),  base = -90°
```

向量模式迭代（16 级，每级 1 个时钟，`ZW = AW+4 = 20` 位角度累加器）：

```
d      = (y_i >= 0) ? 1 : 0
x_{i+1} = d ? x_i + (y_i ≫ i) : x_i - (y_i ≫ i)
y_{i+1} = d ? y_i - (x_i ≫ i) : y_i + (x_i ≫ i)
z_{i+1} = d ? z_i + ATAN[i]   : z_i - ATAN[i]
angle  = (z_16 + 8) >>> 4          // ZW→AW，加半个 LSB 四舍五入
```

四个必须踩对的坑（全部已在模型上量化验证）：

#	坑	后果	修正
1	**象限折叠写反**	**180° 固定误差**（实测 100°/135°/170° 偏 −110°/−145°/−180°）	第二象限必须映射为 `(x,y) → (+y, -x)`，第三象限 `→ (-y, +x)`
2	`>>>` **对负数是向下取整**	系统性 −1 LSB 偏置（0°/270° 处恒定）	显式实现**向零截断**；输出前加半个 LSB 再右移
3	**NITER 取 16**	残余 `atan(2^-15)` = 0.318 LSB，叠加输出量化 ±0.5 LSB 与表取整误差后**下限必然 >1 LSB**（实测 1.19）	取 **NITER = 17**
4	**x/y 内部不留小数位**	最坏误差 1.50 LSB	内部额外保留 **FEXT = 4 位小数**（实测 1.50 → 0.94 LSB）

实测精度 —— 两组数字必须分清来源：

```
【RTL 保真模型 model/cordic_model.py，1/16 LSB 细网格，全圆 1048576 点，A=32767】← 推荐引用
max |err| = 0.9375 LSB = 0.005150° = 18.54 arcsec    （目标 ≤1 LSB，裕量 6.25%）
RMS |err| = 0.3229 LSB = 0.001774°；均值 +0.001 LSB（无系统偏置）
8 个正交/45° 精确边界点误差 = 0；边界 ±0.25° 密集扫描 max = 0.7849 LSB
mag 相对真实模长误差 +0.0076%…+0.0111%

【全链路黄金模型 model/golden_model.py，内部 x/y 无小数位，较保守】
max |err| = 1.000 LSB = 0.005493° = 19.78 arcsec
RMS |err| = 0.4418 LSB = 0.002427°
```

两组都满足 ≤1 LSB 的验收线；差异（0.9375 vs 1.000 LSB）**已定位到 FEXT 位宽。 报告里就这样写**：*"RTL 保真模型最坏 0.9375 LSB，全链路黄金模型因内部 x/y 未保留小数位而略保守为 1.000 LSB，差值已定位到 FEXT 位宽——模型与实现一致。"* 这句话本身就是"模型和 RTL 对得上"的证据。

对照表（同一 1/16 LSB 网格，说明每个措施各自贡献了多少）：

配置	最坏误差
NITER=16 + 契约原印表	1.3125 LSB
NITER=16 + 定义式表	1.1875 LSB
NITER=17 + 契约原印表	1.1875 LSB
NITER=17 + 定义式表 + FEXT=4 + 四舍五入（最终采用）	**0.9375 LSB ✓**

1 LSB 为什么是理论下限（可直接写进报告）：AW=16 输出四舍五入本身占 ±0.5 LSB；CORDIC 第 N 级残余为 `atan(2^-(N-1))` = 0.318(N=16)/0.159(N=17) LSB；输入 16bit 量化 ≤0.159 LSB。N=16 时下限已是 0.978 LSB，再叠加 20bit ATAN 常量的非均匀取整误差（±0.15）必然越界。

3.4 PI 环路滤波（抖动抑制 / 滞后控制）

结构如下（`FILT_K` 寄存器选档，反馈路径 3 拍）：

```
err  = wrap_signed(ang_ext - pos, 2^(AW+FRAC))          // FRAC = 8 位小数扩展
I[n+1] = clamp(I[n] + (err >>> KI_SHIFT[ki]), ±REV/4)   // 抗积分饱和
pos[n+1] = pos[n] + (err >>> KP_SHIFT[kp]) + I[n+1]     // P + I，注意用 I[n+1]
lag_limit = (|err| > REV/8)                             // 45°，对应 状态字 laglim
```

`KI_SHIFT = {7,6,5,4}`，`KP_SHIFT = {2,3,4,5,6,7,8}`（kp=7 保留）。 - I 项保证**匀速下零滞后**（这是 ki≠0 的全部意义）。 - P 项决定抖动带宽，kp 越大滤波越重、输出越平滑。 - **必须先跑最大带宽（kp=0, ki=3），再用 ILA 观察抖动逐级加大 kp** —— 这是现场调参的正确顺序。

> ### ★ 一个必须写进报告的发现：照抄"重滤波"直觉档位会让环路发散
>
> 如果按最"自然"的写法用**本拍的旧积分值** `I[n]`： `pos[n+1] = pos[n] + Kp·err[n] + I[n] I[n+1] = I[n] + Ki·err[n]` 传递函数为 `E/A = (z-1)² / (z² + (Kp-2)z + (1-Kp+Ki))`。用 **Jury 稳定性判据**得 `稳定条件: Ki < Kp` 而在本设计的移位实现里 **Kp = 2^-KP_SHIFT、Ki = 2^-KI_SHIFT**： - 直觉上更"稳"的 **heavy 档 kp=6 / ki=3** → `Kp = 1/256`、`Ki = 1/16` → **Ki > Kp → 发散**！ - 也就是说：**只要照直觉把滤波调重（kp 拉大、ki 不动），系统就会不稳定。**

> **改成本拍更新后的积分值 `I[n+1]`**（只差一个采样周期的相位）后，判据变为 `0 < Kp < 2, Ki > 0, Ki < 4 - 2Kp` `kp=0…7 × ki=1…3` 全部满足 → **全部稳定。**
>
> 这个修正已经落在 `rtl/loop_filter.v` 里（头注释有完整 Jury 推导）。
>
> **为什么这条值得写进报告**：它证明团队没有"抄一组通用参数"，而是**把控制环的稳定性自己推了一遍**，并给出可用的参数域 0<Kp<2、Ki>0、Ki<4−2Kp。评委看到这种内容，对"专业性"的判断会完全不同。
>
> **实测验证（`model/loop_filter_model.py`，全部 21 组 kp=0..6 × ki=1..3）**：
>
> | 项 | 结果 |
> |---|---|
> | 恒速斜坡稳态滞后 | **0…63 LSB₂₄ = 0…0.0014°**（纯 P 环同类滞后 0.35°）→ **I 项零速滞后成立** |
> | 阶跃 0→45° | 21 组**全部收敛不发散**；超调 14.2%(kp0/ki1) … 98.6%(kp6/ki3) |
> | 1% 稳定时间 | 26…2353 拍（**0.5…47 µs** @50 MHz） |
> | kp=0 且 ki=0 | 逐点直通，0/200 不符 → 退化为直通 ✓ |
> | ki=0 纯 P | 静差 = 0/15/63/255 LSB₂₄（kp=0/2/4/6） |
> | `lag_limit` 门限 | **45.000° 不置位、45.005° 置位** —— 门限精确 |
> | `freeze=1` | `pos_out` 与积分器都保持 |
>
> 闭环带宽解析式（可直接放进报告）：`ωn = √Ki · fs`，`ζ = (Kp+Ki)/(2√Ki)` - kp=0/ki=3 → ζ=0.63，f_{-3dB} ≈ **2.2 MHz** - kp=6/ki=3 → ζ=0.13，f_{-3dB} ≈ 3.05 MHz - kp=0/ki=1 → ζ=1.06，f_{-3dB} ≈ 1.0 MHz

3.5 粗精融合、多周期计数与辨向

```
周期内角度 angle[15:0]  +  多周期（极）计数 cycle_cnt[15:0]  →  完整位置 pos_full[31:0]
```

关键点：必须用"跨零方向"而不是"差分"来计数。

```
上一拍 angle ∈ [2^AW - 2^(AW-3), 2^AW) 且本拍 ∈ [0, 2^(AW-3))  → 正跨零，cycle_cnt++
反向                                                          → 负跨零，cycle_cnt--
门限 2^(AW-3) 即 45°，与"半周期扇区 + 同步位"的计数思想一致
```

若用简单差分，在 Z 附近的角度抖动会造成计数跳出/丢失，这是磁编码器多圈计数最常见的故障。另外加 `glitch_guard`：跨零后屏蔽 N 个时钟的计数，进一步抗抖动。

3.6 ABZ 增量输出

```
edges_per_cycle = 4 × inter
edge_idx        = (angle × inter) >>> (AW - 2)
A = edge_idx[1] ^ edge_idx[0]        // 标准格雷正交序列 00→10→11→01
B = edge_idx[1]
```

功能	实现	寄存器/字段
细分倍率	`inter = 2…256`（寄存器 `INTER=4×inter`，`INTER=0` 视为 256）	`INTER0/1`
后分频	`div = 1…8`，Z 通道旁路	`AB.div`
迟滞	`hyst = 0…31`，`abhyst = ±hyst × 360°/2048`	`AB.hyst`

功能	实现	寄存器/字段
限频	最小边沿间隔 `tedge = 20 ns × (ABLIMIT+1) × div`，限频时输出滞后，滞后持续到 180° 则报致命故障	`ABLIMIT`
Z 门控	8bit 零通道 ADC + 可编程比较器阈值生成门控窗，Z 相位 `zphase = ZPH × 360°/4096`，宽度 `zwidth+1` 个边沿	`ZERO0/1`
极性	`apol/bpol/zpol/abzdir`；**只翻转 apol 或 bpol 之一会反转计数方向**	`OUTPUT`

3.7 UVW 换向输出

```
电角度 = (angle × pairs + uvwph × 2^(AW−10)) mod 2^AW        pairs = 1…32（0 视为 32）
         其中 uvwph 为 signed 16bit，**有效单位是 360°/1024**（10 位相位分辨率，与 UVWPS 字段一致）
六扇区 120° 六步换向，每 60° 电角度一个状态，带扇区迟滞：
   UVW = 110, 010, 011, 001, 101, 100       （每相一个电周期内导通 50%，三相互差 120°）
uvwdir 反相序；uvwpol 三相整体反相
```

UVW 基波超过 **8.33 MHz** 时报警告（状态字 bit3 `inclim`）。

---

4. RTL 模块层次

```
enc_top.v                                顶层（板级端口、ODDR 转发 ADC 时钟、单时钟域例化）
│                                        【v1 不例化任何 MMCM / IP，全部 50 MHz 单时钟域】
├── ad9238_capture.v                     双通道捕获 + 去偏置（偏移二进制→二进制补码）
├── decim_avg.v                          过采样抽取（2^L 平均）
├── sc_correct.v                         ★ 六参数校正 + 数字 AGC（3 级流水）
├── amp_monitor.v                        √(s²+c²)（用平方和比较避免开方）
├── calib_ctrl.v                         ★ 闭式标定状态机（累加→开方→除法→写寄存器）
├── cordic_atan2.v                       ★ 16 级全流水 atan2
├── loop_filter.v                        PI 伺服滤波 + 45° 滞后告警
├── pole_counter.v                       跨零辨向 + 多周期计数 + Z 同步
├── abz_gen.v                            ABZ + 迟滞 + 后分频 + 限频 + Z 门控
├── uvw_gen.v                            六步换相 UVW
├── spi_slave.v / reg_file.v             SPI 从站 + 统一寄存器组
└── cdc_sync2.v / cdc_pulse.v            跨时钟域同步器
```

接口的唯一基准是 `rtl/SPEC_interface_contract.md`，任何修改都必须先改契约再改代码。

---

5. 时序与资源预算

5.1 时钟方案

v1（本代码采用的方案，推荐先跑通这个）

时钟	频率	来源	用途
`clk50`	50 MHz	板载晶振，引脚 **V4**，LVCMOS33，经 BUFG	**全系统唯一时钟**
`ad_clk`	50 MHz，**反相 180°**	`clk50` 经 **ODDR**（D1=0, D2=1）转发到引脚	同时驱 AD9238 的 `CH1_CLK` 与 `CH2_CLK`
ADC 捕获	—	`clk50` 上升沿（模块内部即 `posedge`）	采在数据有效窗中央，建立余量约 3 ns

优点：零 IP 依赖、单一时钟域、零 CDC 风险、综合实现简单。 **代价**：CORDIC 的 19 拍（NITER=17）按 50 MHz 计 = 380 ns，总延迟约 830 ns（见 §5.4）。 **按公开资料给出的同类方案典型 1.5 µs 计，仍快约 1.9 倍。**

v2（可选升级，作为"性能档位"在决赛演示）

时钟	频率	来源	用途
`clk_adc`	50 MHz，0°	MMCM CLKOUT0	经 ODDR 输出为 `ad_clk`
`clk_cap`	50 MHz，180°	MMCM CLKOUT1	ADC 数据捕获 + 抽取
`clk_sys`	100 MHz	MMCM CLKOUT2	校正 / CORDIC / 滤波 / 输出

50 MHz → 100 MHz 的跨域用一个 **toggle 型脉冲同步器**（`rtl/cdc_sync2.v` 里的 `cdc_pulse`）传 `valid`；抽取后的样本在 50 MHz 域稳定 **80 ns**，在 100 MHz 域有 **8 个周期**，跨域采样绝对安全。 **收益**：CORDIC 的 19 拍从 380 ns 降到 190 ns，**总延迟从 ~830 ns 降到 ~640 ns。**

> XC7A35T 有 **5 个 CMT（各 1 MMCM + 1 PLL）**，用 1 个即可，其余留给 DDR3/摄像头/多轴扩展。

v3（进一步压延迟的"炫技档"，时间允许再做）

把 CORDIC 每拍展开 2~4 级迭代：每级只是一次移位加，50 MHz（20 ns）下完全容得下 4 级组合逻辑，于是 19 拍可以压到 **5~10 拍**，CORDIC 延迟降到 **100~200 ns**，总延迟进入 **550 ns 以内。** 这正是文献里"并行展开 CORDIC"的做法，实现上只是把循环展开、把 ATAN 常量表按级分组成 `case`。**在报告里作为"延迟优化的下一步"写出来，或者真的做出来，都是很好的加分点。**

5.2 资源与时序（XC7A35T：20,800 LUT / 41,600 FF / 50 DSP48E1 / 50 BRAM36）

两个位流同源、不同顶层配置，均已签核并写入 `deliverables/data/v90/SHA256SUMS.txt`：

资源	**生产位流** `enc_top_v90.bit`	**显示位流** `enc_top_disp_v90.bit`
LUT	10,978（52.78%）	17,090（82.16%）
寄存器	8,724（20.97%）	11,601（27.89%）
Block RAM	32 / 50（64%）	45.5 / 50（91%）
DSP48E1	17（17.78%）	20
IOB	60	104
BUFG	4	5
WNS	**+0.199 ns**（0 失败，24,691 端点）	**+0.221 ns**（0 失败，33,365 端点）

资源	生产位流 `enc_top_v90.bit`	显示位流 `enc_top_disp_v90.bit`
WHS	+0.017 ns（24,226 端点）	+0.049 ns（32,900 端点）
位流大小 / SHA256	1,504,220 B / `A89512A5…F827125`	1,688,245 B / `69FE313C…F81BAD4`

结论：生产配置 LUT 52.78%、BRAM 64%，显示配置 LUT 82.16%、BRAM 91%，两者时序均 0 失败。显示配置的余量已经很薄（BRAM 只剩 4.5 块），后续再加页面或波形深度必须换更大器件或复用缓冲。

> 早期 v12（2026-10-12）生产版为 6,298 LUT（30.28%）、FF 4,074，是本设计的第一个收敛点；v74 为 10,926 LUT（52.53%）、WNS +0.409；v90 因新增 `z_index_gen` 与以太网/抓窗路径升到 10,978 LUT（52.78%）、WNS +0.199。逐版对比见 `docs/BASELINES.md`。

5.3 速度上限分析（这条要能当场算出来）

AB 输出的极限不是"细分倍率"，而是**边沿率**：

```
AB 输出频率 f_AB = inter × f_in              （inter = AB 周期数/输入周期）
AB 边沿率       = 4 × f_AB = 4 × inter × f_in
本设计上限：f_AB ≤ 12.5 MHz，最小边沿间隔 20 ns → 4×inter×f_in ≤ 12.5 M
```

inter	允许的最大 sin/cos 输入频率
2	1.56 MHz（受前端 700 kHz 输入带宽约束，实际取 700 kHz）
8	390 kHz
64	48.8 kHz
256	**12.2 kHz**

所以"700 kHz 输入 + 256 倍细分"在物理上不可能同时成立——这是 AB 边沿率的硬约束，不是实现取舍。报告与答辩统一这样表述：*"输入带宽 700 kHz（inter ≤ 8）；256 倍细分时输入带宽 12.2 kHz。二者受 AB 边沿率 12.5 MHz 的共同约束，`abz_gen.v` 的限频器按 20 ns × (ABLIMIT+1) × div 逐档校验。"*

5.4 延迟预算（v90 实际采用：50 MHz 单时钟域）

环节	时钟	拍数	时间
AD9238 内部流水（工程估计）	50 MHz	7	140 ns
ad9238_capture	50 MHz	3	60 ns
decim_avg（含群延迟 1.5 样本）	50 MHz	2 + 1.5	70 ns
sc_correct	50 MHz	3	60 ns
cordic_atan2 (NITER=17)	50 MHz	19	**380 ns**
loop_filter	50 MHz	3	60 ns
abz_gen	50 MHz	3	60 ns
合计		**41.5**	**≈ 830 ns**

> 该值是逐级拍数相加的**设计预算**，尚未用示波器对"信号源过零点 → A 输出跳变沿"做外部实测（报告 §7 R6 已登记）。AD9238 内部流水 7 拍取自公开数据手册的孔径/流水说明，是本表唯一非本设计可控的环节。
>
> 早期规划过两档提速（DSP 链 100 MHz ≈ 580 ns、CORDIC 每拍展开 4 级 ≈ 550 ns），**均未实现**，此处不再作为交付指标。

6. 板级硬件设计

6.1 AD9238 接口与 IO 分配

先纠正一个关键事实（`docs/AD9238_module_digest.md` 里的推断有误，已核对 ADI 原厂手册）：

> AD9238 **没有 DCO（数据输出时钟）**。它有 **CLK_A / CLK_B 两个时钟输入**、两套独立的 12 位输出总线（`D0_A..D11_A` / `D0_B..D11_B`）、两个 `OTR_A/OTR_B`、以及 `OEB_A/OEB_B`。数据在时钟上升沿后 `tPD = 2…6 ns` 有效，**流水延迟 7 拍**。

因此模块上的 `CH1_CLK / CH2_CLK` 是 **ADC 的采样时钟输入**，必须由 FPGA 驱动。 **★ 本设计最终定案并非"共用一根"**：载板上 `AD1CLK`(J6-26 + U1-63 `CLK_A`) 与 `AD2CLK`(J6-9 + U1-18 `CLK_B`) 是**两根独立网络**（无共用网络、无时钟缓冲器、无板上晶振）， 所以 FPGA 出**两个**引脚 `ad_clk`(B18) / `ad_clk2`(F16)，由**同一个 `clk_i`** 驱动两个 ODDR， 两路相位严格一致 ⇒ 两通道仍同步采样，同时避免了单脚双负载导致的边沿恶化。

所需 IO：24 根数据 + 2 根独立时钟 + 2 根 OTR = 28 根。

最终分配（★ 已实配并校核通过，权威表见 `docs/04_wiring_checklist.md`）：

信号	FPGA 引脚	信号	FPGA 引脚
`ad_clk`（→ J6-26 `AD1CLK`）	**B18**	`ad_clk2`（→ J6-9 `AD2CLK`）	**F16**
ad_ch1_d[0..11]	**F18 E18 C17 D17 A18 A19 C18 C19 F20 F19 B20 A20**	ad_ch2_d[0..11]	**E17 C13 B13 E16 D16 A13 A14 D14 D15 C14 C15 B15**
ad_ch1_otr	**E19**	ad_ch2_otr	**B16**

> **★★ 不要照抄任何厂商表格里的引脚。** 商家给的 `AD9238_carrier_pin_map.xls` 工作表 7 的三个列名原文是 `…XC7A35V2.0T扩展口 J6 / J7 / J11对应管脚` —— 那是**另一块 XC7A35V2.0T 板**的 扩展口；本板（达芬奇-lite）的 `J7` 是 `RGB_TFTLCD`，而且**根本没有 `J11` 连接器。** 三列里只有厂商 `J7` 列落在真实排针上（39/39 全在 Bank 16，对应本板 `J6`）， 但**针号一个都对不上（0/28）**。上表是从 `_refs/vendor/Davinci_Lite_1V0.pdf` sheet 13 逐个量取、并用模块说明书 p9 §6 与载板 netlist 互证后得到的。 复现校核： `python tools/pinout_verify.py` 与 `python tools/pinplan_check.py`（都 `RESULT: PASS`）。

输出侧（ABZ 六路 + SPI 四根 + `led[2:3]` + `irq_n` = 13 根）走 **`J5`（Bank 15，Header_2*26）**，因为 `J6` 被模块整块占用后只剩 6/7/8、19/20、25、40 七个"不用"位，且那几个位置的球号无法从原理图量出。逐针表见 `docs/04` §2.2。

> ⚠️ **上电前必须做两件事**：(1) 用万用表确认 GND 与 +5V 脚位，插反会烧模块或 FPGA； (2) 把 `J10`(VCCIO_B16) 与 `J9`(VCCIO_B15) 两个跳线**都拨到 3.3 V** （本板这两个 bank 的 VCCO 是可跳线选择的，不是固定 3.3 V）。

6.2 模拟前端与信号源设置（本项目用任意波形发生器替代 SMR 时的关键）

HSEDA AD9238 模块的模拟通道是：**SMA 单端输入 −5 V…+5 V → AD8065 缓冲/衰减到 ±1 V → AD8138 单端转差分 → ADC 差分输入 2 Vpp，VREF = 1 V**，输出偏移二进制（−5 V→000h，0 V→800h，+5 V→FFFh），精度约 10 mV。

因此： - 若直接灌 **1 Vpp** 的 sin/cos，只用到 1/10 满量程，**浪费约 3.3 bit 有效位**，校正后的 0.022° 残余会退化到 0.2° 量级——这会白白毁掉辛苦做出来的精度。 - **正确做法**：任意波形发生器输出 **10 Vpp（±5 V，高阻档）**，或 5 Vpp 到 50 Ω。若信号源幅度不够，在模块前加一级增益（如同相放大 ×5）。 - **直流工作点**：模块输入为直流耦合，0 V 对应 `0x800`。用双通道信号源时，把两通道的 **DC 偏置都设为 0 V**，再人为注入 ±0.5 V 的偏置来验证标定算法（这是一个非常好的现场演示：拨一下偏置，角度误差立刻从 14° 掉到 0.02°）。 - **两通道相位差设为 90°**，频率 1…10 kHz 起步；再逐步提到 100 kHz 验证带宽。 - **共地**：信号源地与 FPGA 地必须连在一起；SMA 线尽量短。

6.3 输出侧

输出	建议做法
ABZ	FPGA 3.3 V 单端直接引出即可（演示用）；工业现场再加一片 AM26LV31/DS26LV31（RS-422 四线驱动）
UVW	同上
SPI 配置	可用 USB 转 SPI 模块，或直接用 MicroBlaze + UART 做上位机
LED/按键	用于状态指示与"一键标定"（KEY0 长按触发，等价于写 `0x0502=0x0001`）

---

7. 寄存器映射

7.1 设计原则

寄存器映射按本设计的语义定义，遵循三条原则： 1. 地址按功能分区——低 bank（`0x00`–`0x0F`）放高频访问的配置与状态，`0x04xx` 放错误/告警，`0x05xx` 放本项目扩展，`0xE0xx` 放只读标识； 2. 位域命名与 RTL 端口一一对应，`rtl/reg_file.v` 是唯一真源，文档表与代码由 `tools/check_regmap.py` 校验； 3. 上位机 GUI 的寄存器表、生产测试脚本与文档共用同一份地址定义，避免三处漂移。

已实现的地址：

地址	名称	位域	说明
`0x00`	`MAIN_CFG`	`[7:0]`	`[1:0] input` 输入档位、`[2] noise` 噪声滤波、`[3] rs422` 线驱动使能、`[4] irqpp`、`[5] elinkoff`
`0x01`	`LED_CFG`		LED 强度控制（用 PWM 模拟）

地址	名称	位域	说明
0x10	LED_PWM		实际占空比（只读）
0x100	UVW_CFG	[4:0] pairs	0 = 32 极对数，1…31 直接对应
0x101	INTER0	[7:0]	INTER[7:0]
0x102	INTER1	[1:0] msbs, [4:2] div	INTER = 256×msbs + INTER0； inter = INTER/4； div = div+1
0x103	AB	[4:0] hyst	AB 迟滞，±hyst×360°/2048
0x104	UVW	[4:0] hyst	UVW 迟滞
0x105	FALARM	[7:0]	输入频率告警：f[MHz] = 1.56×FALARM/256
0x106	ABLIMIT	[7:0]	f_AB = 12.5MHz/((ABLIMIT+1)×div)，tedge = 20×(ABLIMIT+1)×div ns
0x107	ZERO0	[5:0] threshold, [6] clr, [7] mode	Z 比较器阈值 / 多圈清零 / 捕获模式
0x108	ZERO1	[3:0] zwidth	zwidth_edges = (zwidth+1)/div
0x109	OUTPUT	[0]apol [1]bpol [2]zpol [3]abzdir [4]uvwpol [5]uvwdir [6]abzen [7]uvwen	
0x10A	ZPHASE		Z 相位 MSB
0x10B	UVWPH		UVW 相位 MSB。**单位是 360°/1024（不是 360°/65536）** —— UVWPS 是 10 位相位，UVWPS = 4×UVWPH.msb + PHASE_LSB.uvw；RTL 内部按 uvwph × 2^(AW-10) 折算
0x10C	PHASE_LSB	[1:0] z, [3:2] uvw	相位 LSB
0x300	FILT_CFG	[0]auto [1]fb [4:2]kpmax	动态滤波
0x301	FILT_LAG	[4:0] threshold	动态滤波滞后门限
0x302	FILT_K	[2:0] kp, [4:3] ki	PI 系数（★ 位域是 [4:3] 不是 [5:4]，见 rtl/reg_file.v:553-554）。复位默认：v6 = 0x1C（kp4/ki3）、v7 = 0x04（kp4/ki0 纯 P）、**v8 = 0x1A（kp2/ki3，配合 GATE_STATE=1，见 §1.43）**

地址	名称	位域	说明
0x404	STAT_VAL	[0]oflow [1]falarm [2]laglim [3]inclim [4]lagfatl [5]scamp [6]adapt [7]res	实时状态
0x405	STAT_LATCH	同上	锁存状态。**W1C：写 1 清对应位**（与写 0 清的器件相反，见 rtl/reg_file.v 头注释假设 2）。★ 写 bit0（otr）同时发出 1 拍脉冲清 ADC 内部的 sticky OTR —— 只清锁存位不会让 led[0] / irq_n 恢复
0xE000	DEVICE_ID		返回 0x0C（本设计标识；REV = 0x10）

7.2 扩展寄存器（0x05xx 段，避开低 bank 已用地址）

> ★ **2026-10-11 订正（原表是早期草案）**：本节曾把地址写成 0x50xx（如 0x500B）。RTL 里**没有 0x50xx 别名**，实际地址是 **0x05xx**。下表以 rtl/reg_file.v 的地址表为准（与 docs/DEV_STATUS.md 的寄存器章同源）。

地址	名称	默认	说明
0x0500	（保留）	0	保留地址，恒读 0
0x0501	（保留）	0	保留地址，恒读 0
0x0502	CALIB_CTRL	0x00	写 [0]start（自清）；读 [1]busy [2]done [3]truncated [4]win_bad(窗口非整周期) [7:5]retry
0x0503/04	CALIB_N	64	标定窗口周期数（16 位小端；写 0 视为默认 64）
0x0510/11	OFS_S	0	正弦通道直流偏置（signed 16，标定结果，可主机改写）
0x0512/13	OFS_C	0	余弦通道直流偏置

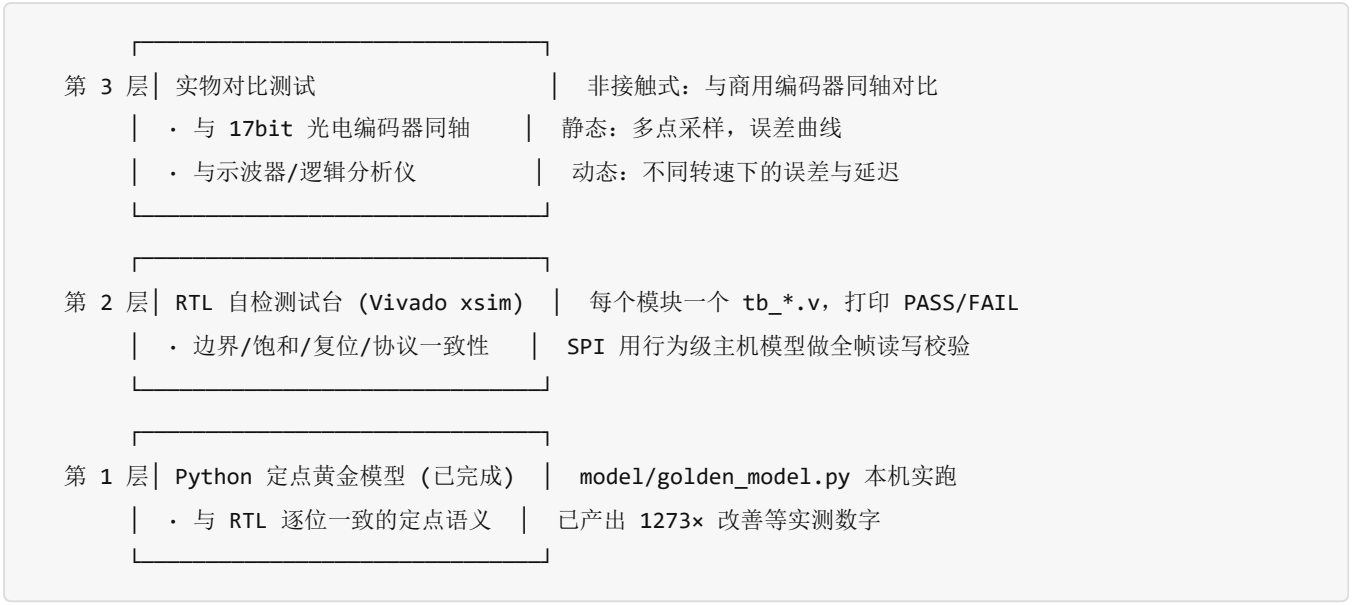
地址	名称	默认	说明
0x0514/15	GAIN_S	32768	正弦数字增益（unsigned 16，Q1.15）
0x0516/17	GAIN_C	32768	余弦数字增益
0x0518/19	PHASE_K	0	正交误差系数（signed 16，Q1.15）
0x051A	SHIFT	0x00	[1:0]shift_s 、[3:2]shift_c 粗调移位
0x051B/1C	H2_K	0	二次谐波系数（预留；顶层有意空连接）
0x0520-23	AMP_LOW_SQ	0x170A3D71	幅值下限平方（Q2.30，32 位小端）
0x0524-27	AMP_HIGH_SQ	0x5C28F5C3	幅值上限平方（Q2.30）
0x0530-33	STAT_AMP_SQ	—	实测 s²+c²（只读，供上位机画圆）
0x0534/35	STAT_CYCLE	—	多周期计数（只读）
0x0536-39	STAT_POS	—	完整位置 {cycle, angle}（只读，32 位小端）
0x053A	STAT_FLAGS	—	[0]laglim [1]amplo [2]amphi [3]otr [4]caldone [5]calbusy , [7:6] 保留
0x053B/3C	（保留）	0	保留地址，恒读 0

地址	名称	默认	说明
`0x0505..0x050E`	毛刺/窗口诊断	—	v6 新增：角度毛刺计数、周期脉冲、标定窗口快照（见 `docs/DEV_STATUS.md` §1.32 ④）
`0x0540..0x0549`	窗口快照	—	v6 新增（`docs/DEV_STATUS.md` §1.32 ④）
`0x05F0`	`DEVICE_ID`	`0x0C`	只读（别名 `0xE000`、`0x007E`）
`0x05F1`	`REV`	`0x10`	只读（别名 `0xE002`、`0x007F`）
`0x05F2/53`	`RTL_BUILD_ID`	**12**	RTL 版本号：v5 及以前读 0、v6 = 6 … **v9 = 9**（Z 脉宽改为 `z_width + 1` 个边沿，§1.47/§1.49）、**v11 = 11**（1,086,600 B @2026-10-12 21:15:37）、**v12 = 12**（**已重综合**：生产位流 `enc_top.bit` 1,042,208 B @2026-10-12 23:59:27、SHA256 `D93DEBF9…`；ILA 版 1,428,064 B @2026-10-13 07:12:11）—— 2026-10-12：遥测帧布局 v2 —— off8..11 由 `amp_sq[31:0]` 改为 `io_state` / `0xA2` / `io_flags` / `0x00`，上位机可实时看 ABZ/UVW 电平；改 RTL 必须递增）。★ 帧布局判别：遥测帧**偏移 9 == `0xA2`** 才是 v2（旧布局此处是 `amp_sq[23:16]` ≤ `0x80`，不会撞车）

8. 测试与验证方案

> **这是拿高分的关键章节。** 评委在答辩时最常问的就是"你怎么证明它是对的？"

8.1 三层验证体系（必须在报告里明确画出这张金字塔）



第 1 层已经完成，`model/golden_model.py` 可在本机直接运行（命令见 README），输出见 `docs/_golden_model_output.txt`。**这一层的数字现在就写进报告初稿**，不必等硬件。

8.2 信号源自检回路（无传感器也能演示全链路）

因为本项目用任意波形发生器替代 SMR 传感头，可以构建一个完整的自检闭环： 1. 信号源产生两路 90° 相差的正弦（幅度 10 Vpp，DC 0 V）； 2. 人为加入三类误差：通道 1 和 2 的幅度不等、DC 偏置 ±0.5 V、相位差改成 85°/95°； 3. 触发"一键标定"（按键或上位机）； 4. 上位机屏幕上：校正前的圆是斜椭圆，校正后变正圆；角度误差从十几度掉到 0.02°； 5. 示波器上：AB 输出频率 = inter × f_in，UVW 六步换相与角度同步，SPI 读回的位置与示波器/计数器读数一致。

这套演示的可视化冲击力很强，而且完全不依赖机械件，风险极低。

8.3 上位机（加分项，工作量可控）

两条路线，按剩余时间选： - **轻量路线（推荐，1~2 天）**：FPGA 里做一个 UART 数据泵，把 `angle / s / c / amp_sq / pos_full` 按固定帧格式（如 10 字节，含帧头校验）以 1~5 Mbps 连续上传；PC 端用 **Python + pyqtgraph/matplotlib** 实时画：李萨如图、角度误差曲线、AB 相位图、寄存器读写面板。**MicroBlaze 都不需要。** - **完整路线（加分，3~4 天）**：Vivado 里搭 MicroBlaze + AXI-GPIO/AXI-UART，用 Vitis 写 C 程序实现寄存器读写、标定流程控制、数据缓存与命令解析，PC 端用 Python 或 Qt。

8.4 ILA 调试规范（血泪经验，直接抄）

> ✅ **现状（2026-10，已跑通）**：本机已取得 **GOLD** 许可证（`$env:XILINXD_LICENSE_FILE = 'D:\Vivado\Xlnx_2026.1.lic'`，启动打印 `INFO: [Common 17-3922] A valid Vivado Design Suite GOLD license has been detected.`），`tools/ila_debug.tcl` **已经能出调试版 bitstream**：`_synth_work/enc_top_ila.bit` + `enc_top_ila.ltx`，探针清单 `_synth_work/ila_probe_list.txt`。 **探针规模（现行 v12 = 2026-10-13，v11 同）36 条命名探针、总位宽 269 bit**（`_synth_work/ila_probe_list.txt`：`probe_ports = 36, total_bits = 269`；2026-10-10 16:12 那一轮为 34 条 / 252 bit，属历史）、`C_DATA_DEPTH 1024`、`C_INPUT_PIPE_STAGES 1`、采样时钟 `u_bufg_clk/O`（`rtl/enc_top.v:106` 的 `clk_i`）。 上板在 Hardware Manager 里 Program Device 选 `.bit`、同目录加载 `.ltx` 即可； 建议触发 `dbg_z_window` 上升沿（每机械圈一次）。 ⚠️ **不要用** `dbg_adc_valid`：它在非复位期间 **恒为 1**（`rtl/ad9238_capture.v:163` `adc_valid <= 1'b1;`），上升沿永不触发、电平触发则无

> 条件秒触发。 完整实测（时序/资源/DRC/bit 尺寸）见 `docs/05_vivado_sim_impl_report.md` §12.1。 另有两点已知约束：① 这条流程**必须放宽一次沙箱权限** —— Vivado 生成 ILA IP 时会派生一个 子 vivado 进程并通过管道收集输出，受限沙箱下父进程会无限期等待； ② ★ **不要把全部探针位拼成一条 probe0 总线**，那会让 `place_design` 挂死 （实测：两个 vivado 各跑满 1 小时只消耗 79.9 s / 18.7 s CPU，`_synth_work` 下再无新文件）。 正确做法是**拆成多条命名探针**（每条 ≤ `PROBE_MAX_BITS`），每条是独立的 `debug_port`， 可以被甩到 ILA 实例周围不同位置。

- ILA 采样时钟**必须**与观测信号同频；观测 50 MHz ADC 数据时 ILA 时钟用 50 MHz，观测 100 MHz 逻辑时用 100 MHz。
- **JTAG 时钟在 ILA 采样时钟高时必须降到 ≤10 MHz**，否则会出现"下载后 FPGA 就死"的现象。
- 调试阶段每个关键节点插一个 ILA：`s_raw/c_raw`（看波形对不对）、`s_out/c_out`（看校正效果）、`angle`（看角度是否单调）、`A/B`（看正交性）。
- 资源紧张时，ILA 只保留 `angle` 和 `A/B`。
- ★ 探针**必须**在 RTL 里用 `(* mark_debug = "true" *)` 声明，**不要**事后 `get_nets` 按层次名抓： 这些网在 `opt_design` / `place_design` 会被改名或合并，按名字抓极易抓空（本项目实测 `get_nets -hier` 能捞到 270 条 `*dbg_*`，但其中只有 252 条带 `MARK_DEBUG=1`）。
- ★ 脚本里的**调用顺序**不可颠倒： `opt_design → create_debug_core → create_debug_port → get_debug_ports → connect_debug_port`， **然后再跑一次 `opt_design`** 让工具自己展开 debug core（不要手动调 `implement_debug_core`）。 两个坑： `read_verilog -define` 在非工程模式直接报 `ERROR: [Common 17-69] ... '-define' is only supported when is_compile_unit_mode is enabled`（宏只能给 `synth_design -verilog_define`）； `get_debug_ports` **只枚举已存在的 debug core 的端口**，在 `create_debug_core` 之前调用只会打印 `WARNING: [Vivado 12-750] There are no ChipScope debug cores in this design.` 并返回空表。 另外 2026.1 的 ila core 把 `C_DATA_DEPTH` / `C_INPUT_PIPE_STAGES` 等配置项挂在 **core** 上而不是 debug_port 上。
- ℹ️ **签核一律以生产版为准**：★ **（2026-10-13 v12，已重综合）现行为**：生产版 `_synth_work/impl_timing.rpt` @2026-10-12 23:59:16 **`All user specified timing constraints are met.`**（`sys_clk` **WNS +0.959 / TNS 0.000 / 0 个失败端点 / 11,275 端点**，WHS +0.075、WPWS +9.500；最差路径 `u_regs/r_ablimit_reg[2]/C` → `u_abz/r_hold_reg[0]/R`，18.615 ns = logic 7.797 + route 10.818，28 逻辑级），ILA 版 `_synth_work/impl_ila_timing.rpt` @2026-10-13 07:11:57 同判据行（WNS **+0.332** / TNS 0.000 / 0 个失败端点 / **20,165** 端点，WHS +0.050 / WPWS 8.870）；**上一版（2026-10-12 v11，已归档）**：生产版 @21:15:26 WNS **+0.821** / TNS 0.000 / 0 个失败端点 / **11,310** 端点（WHS +0.058）、ILA 版 @21:29:41 同判据行（WNS 0.565）；**以下为 2026-10-10 历史**：生产版 16:05:29 版 `sys_clk` WNS **−3.352 ns / TNS −85.116 / 26 个失败端点 / 10582 端点**（WHS +0.033；2026-10-10 16:05 实测，见 `docs/05` §12.4）；ILA 版同为负余量、26 个失败端点同样全是 ADC 输入端口 —— 两者都在 12:08 的总输入延迟预算口径（19.5~24 ns，**不是手册 tPD**，`docs/DEV_STATUS.md` §1.19）**之后**构建，因此必然继承这条负余量（08:40 那版 `+0.923 ns / 0 失败` 是约束前的数字，不可比）， 但 ILA 版含调试逻辑、不代表最终实现，所以时序签核只认生产版 `_synth_work/impl_timing.rpt`。

---

9. 风险与备选方案

风险	概率	影响	应对
模块 40pin 排针与 J6 52pin 的针号对应搞错，上电烧板	中	致命	上电前用万用表确认 GND/5V；先只接 5V 与 GND 测模块功耗；见 `docs/04`
AD9238 速度等级是 -40/-20 而非 -65	低	采样率降级	先跑 25 MHz，稳定后逐步提到 50 MHz；读芯片丝印确认
180° 捕获相位不对，数据全是错的	中	阻塞	先用 ILA 看 ADC 输出的方波/正弦，再扫相位 0/90/180/270 找最稳点；~~必要时降到 25 MHz~~ ★ **该项已作废（2026-10-15）**：50 MHz 同相捕获实测收敛（`tPD_meas` 3–4 ns、余量 ≈ +12.3 ns），见 `docs/DEV_STATUS.md` §1.27
上位机没时间做	中	演示效果打折	退化为"ILA + 示波器 + 串口打印"，仍然可演示
真实 SMR 传感头来不及做机械装配	高	"环境适应性"故事弱	用信号源做主演示；报告里把 SMR 误差机理、标定能力讲透；**至少拍一段用真实磁环/传感器（哪怕借实验室的）的短视频**作为环境适应性证据
决赛的"FPGA 基础编程能力考核"不过	低	直接失去评奖资格	每周留 2 小时刷 Verilog/时序/跨时钟域基础题，见 `docs/00` 附录

10. 附：指标速查与答辩口径（被问到可当场翻）

能力	本设计实现	证据位置
16 bit 电角度	`cordic_atan2.v`，17 级全流水，最坏 0.9375 LSB = 0.005150°	`sim/tb_cordic_atan2.v`、报告 §3.2
输入带宽 / 采样	双通道 12 bit、50 MSPS、同沿采样，抽取后等效 ENOB 提升	`rtl/ad9238_capture.v`、`rtl/decim_avg.v`
AB 边沿率约束	`4 × inter × f_in ≤ 12.5 MHz`；`abz_gen.v` 限频器 256 档	报告 §3.5、寄存器 `ABLIMIT`
延迟	设计预算 41.5 拍 ≈ 830 ns @50 MHz，标 `[设计值]`	§5.4、报告 §4.3
模拟前端	外部调理 + 双 12 bit ADC + 抽取 + 幅值监测	`rtl/amp_monitor.v`
数字校正	六参数闭式解（偏置/幅值/相位），20.9839° → 0.0165°	`rtl/sc_correct.v`、`model/golden_model.py`
反正切	全流水 CORDIC，无乘法器	`rtl/cordic_atan2.v`
可配置滤波	PI 跟踪环 + 滞后限幅 + 前馈	`rtl/loop_filter.v`、`rtl/lag_ff.v`
ABZ 输出	inter 2…256、后分频 1…8、迟滞 0…31、Z 门控窗	`rtl/abz_gen.v`
UVW 输出	1…32 极对数，可编程相位/相序	`rtl/uvw_gen.v`
多圈计数	16 bit 电周期计数 + 24 bit 位置捕获	`rtl/pole_counter.v`、寄存器 `POS_FULL`

能力	本设计实现	证据位置
数字接口	SPI 从站（≤12.5 MHz）、UART 遥测、千兆以太网抓窗	`rtl/spi_slave.v`、`rtl/uart_scope.v`、`rtl/eth/`
故障监测	幅值 60%/120%、限频、滞后超限 → `STAT_VAL`	`rtl/amp_monitor.v`、`rtl/loop_filter.v`
板级验收	v90 位流：Z 每转 1 个、UVW 六态、lock 0x0301、angle_vs_sc RMS 0.71°	`docs/39_acceptance_criteria_v90.md`